

RFC-0005 - Modelo de Dados Detalhado (DDL Proposto + Estratégia de Migração)

Status: Draft

Autor: Arquitetura

Data: 2026-02-24

Depende de: RFC-0001, RFC-0002, RFC-0003, RFC-0004

1. Contexto

Este RFC define:

- Modelo relacional inicial.
- Estrutura de tabelas do MVP.
- Índices obrigatórios.
- Estratégia de versionamento de schema.
- Preparação para evolução multi-tenant.
- Estratégia de migração futura (schema por tenant / banco por tenant).

Banco alvo: **PostgreSQL**.

2. Princípios de Modelagem (Diretrizes Arquiteturais)

- **Auditoria:** Preparar colunas para auditoria desde o início: criado_por, criado_em, atualizado_por e atualizado_em operando em UTC.
- **Biometria:** Deve ser referenciada de forma relacional (UUID) na tabela de eventos, com os vetores (templates) armazenados em colunas BYTEA.
- **Defesa em Profundidade:** Constraints do banco de dados (CHECK, etc.) são a última linha de defesa; nunca confiar apenas em validações de aplicação.
- **Imutabilidade Real:** Tabelas como registro_presenca e log_auditoria são estritamente imutáveis. Garantir via TRIGGERS que bloqueiem UPDATE/DELETE ou via revogação de permissões (REVOKE).
- **Multi-tenancy:** Toda tabela multi-tenant deve conter organizacao_id, com exceção de tabelas de domínio global quando aplicável.
- **Performance:** Índices devem refletir os padrões de acesso reais estipulados no MVP.

3. Convenções

- Chaves Primárias:** UUID.
- Timezones:** criado_em e atualizado_em operam em UTC.
- Timestamp Seguro:** recebido_no_servidor_em é obrigatório para registros críticos e representa a hora oficial do servidor.
- Deleção:** Soft delete apenas onde necessário. Nunca deletar registros de presença.
- Nomenclatura Explícita:** Todas as constraints (FKs, UQs, CHECKs) devem ser nomeadas explicitamente para facilitar a manutenção e versionamento pelo Flyway.

4. Esquema de Banco de Dados Físico

Abaixo encontra-se a transcrição exata das estruturas implementadas no banco de dados, compondo o modelo oficial.

4.1. organizacao

```
CREATE TABLE organizacao (  
  id UUID PRIMARY KEY,  
  nome VARCHAR(255) NOT NULL,  
  status VARCHAR(32) NOT NULL DEFAULT 'ATIVO',  
  criado_por VARCHAR(64) NOT NULL,  
  criado_em TIMESTAMP NOT NULL,  
  atualizado_por VARCHAR(64),  
  atualizado_em TIMESTAMP  
);  
CREATE INDEX idx_org_criado_por ON organizacao(criado_por);
```

4.2. unidade

```
CREATE TABLE unidade (  
  id UUID PRIMARY KEY,  
  organizacao_id UUID NOT NULL,  
  nome VARCHAR(255) NOT NULL,  
  tipo_geometria VARCHAR(16) NOT NULL,  
  centro_geo_lat DOUBLE PRECISION,  
  centro_geo_lng DOUBLE PRECISION,  
  raio_geo_metros INTEGER,  
  poligono_geo JSONB,  
  criado_por VARCHAR(64) NOT NULL,  
  criado_em TIMESTAMP NOT NULL,  
  atualizado_por VARCHAR(64),  
  atualizado_em TIMESTAMP,  
  FOREIGN KEY (organizacao_id) REFERENCES organizacao(id)  
);  
CREATE INDEX idx_unidade_org ON unidade(organizacao_id);
```

4.3. vinculo_usuario

```
CREATE TABLE vinculo_usuario (  
  id UUID PRIMARY KEY,  
  organizacao_id UUID NOT NULL,  
  usuario_id VARCHAR(64) NOT NULL,  
  papel VARCHAR(32) NOT NULL,  
  data_inicio timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  data_fim timestamp NULL,  
  status VARCHAR(32) NOT NULL DEFAULT 'ATIVO',  
  criado_por VARCHAR(64) NOT NULL,  
  criado_em TIMESTAMP NOT NULL,  
  atualizado_por VARCHAR(64),  
  atualizado_em TIMESTAMP,  
  FOREIGN KEY (organizacao_id) REFERENCES organizacao(id)  
);  
CREATE UNIQUE INDEX uq_vinculo_usuario_org ON vinculo_usuario(usuario_id,  
organizacao_id);
```

4.4. alocao_unidade

```
CREATE TABLE public.alocacao_unidade (  
    id uuid NOT NULL,  
    vinculo_usuario_id uuid NOT NULL,  
    unidade_id uuid NOT NULL,  
    papel_operacional varchar(32) NOT NULL,  
    status varchar(32) DEFAULT 'ATIVO'::character varying NOT NULL,  
    data_inicio timestamp NOT NULL,  
    data_fim timestamp NULL,  
    criado_por varchar(64) NOT NULL,  
    criado_em timestamp NOT NULL,  
    atualizado_por varchar(64) NULL,  
    atualizado_em timestamp NULL  
);  
CREATE INDEX idx_alocacao_vinculo ON public.alocacao_unidade(vinculo_usuario_id, status);
```

4.5. convite

```
CREATE TABLE convite (  
    id UUID PRIMARY KEY,  
    organizacao_id UUID NOT NULL,  
    unidade_id uuid NULL,  
    papel_esperado varchar(32) DEFAULT 'PARTICIPANTE' NOT NULL,  
    codigo VARCHAR(32) NOT NULL,  
    data_expiracao TIMESTAMP NOT NULL,  
    usado BOOLEAN NOT NULL DEFAULT FALSE,  
    criado_por VARCHAR(64) NOT NULL,  
    criado_em TIMESTAMP NOT NULL,  
    atualizado_por VARCHAR(64),  
    atualizado_em TIMESTAMP,  
    FOREIGN KEY (organizacao_id) REFERENCES organizacao(id)  
);  
CREATE UNIQUE INDEX uq_convite_codigo ON convite(codigo);  
ALTER TABLE public.convite ADD CONSTRAINT fk_convite_unidade FOREIGN KEY  
(unidade_id) REFERENCES public.unidade(id) ON DELETE CASCADE;  
ALTER TABLE public.convite ADD CONSTRAINT chk_convite_papel CHECK (papel_esperado  
IN (  
    'PARTICIPANTE', 'MANAGER', 'GESTOR_TITULAR', 'GESTOR_SUBSTITUTO',  
    'ASSISTENTE', 'OWNER'  
));
```

4.6. perfil_biometrico

```
CREATE TABLE perfil_biometrico (  
    id UUID PRIMARY KEY,  
    usuario_id VARCHAR(64) NOT NULL,  
    modelo_biometrico BYTEA NOT NULL,  
    versao_modelo VARCHAR(32) NOT NULL,  
    criado_por VARCHAR(64) NOT NULL,  
    criado_em TIMESTAMP NOT NULL,  
    atualizado_por VARCHAR(64),  
    atualizado_em TIMESTAMP
```

);

4.7. registro_presenca

```
CREATE TABLE registro_presenca (  
  id UUID PRIMARY KEY,  
  organizacao_id UUID NOT NULL,  
  unidade_id UUID NOT NULL,  
  usuario_id VARCHAR(64) NOT NULL,  
  latitude DOUBLE PRECISION,  
  longitude DOUBLE PRECISION,  
  precisao_gps DOUBLE PRECISION,  
  referencia_biometrica UUID,  
  pontuacao_biometrica DOUBLE PRECISION,  
  biometria_valida BOOLEAN,  
  dispositivo_id VARCHAR(128) NOT NULL,  
  modo_registro VARCHAR(16) NOT NULL,  
  usuario_intermediario_id VARCHAR(64),  
  pontuacao_risco DOUBLE PRECISION,  
  indicadores_risco JSONB,  
  status_tecnico VARCHAR(32) NOT NULL DEFAULT 'RECEBIDO',  
  status_administrativo VARCHAR(32) NOT NULL DEFAULT 'PENDENTE',  
  capturado_em TIMESTAMP NOT NULL,  
  recebido_no_servidor_em TIMESTAMP NOT NULL,  
  criado_por VARCHAR(64) NOT NULL,  
  criado_em TIMESTAMP NOT NULL,  
  FOREIGN KEY (organizacao_id) REFERENCES organizacao(id),  
  FOREIGN KEY (unidade_id) REFERENCES unidade(id)  
);  
CREATE INDEX idx_presenca_org_usuario ON registro_presenca(organizacao_id, usuario_id);  
CREATE INDEX idx_presenca_org_tempo ON registro_presenca(organizacao_id,  
recebido_no_servidor_em);  
CREATE INDEX idx_presenca_unidade_tempo ON registro_presenca(unidade_id,  
recebido_no_servidor_em);
```

4.8. dispositivo

```
CREATE TABLE dispositivo (  
  id UUID PRIMARY KEY,  
  identificador_dispositivo VARCHAR(128) NOT NULL,  
  usuario_id VARCHAR(64),  
  nivel_confianca VARCHAR(16) NOT NULL,  
  plataforma VARCHAR(32),  
  criado_por VARCHAR(64) NOT NULL,  
  criado_em TIMESTAMP NOT NULL,  
  atualizado_por VARCHAR(64),  
  atualizado_em TIMESTAMP,  
  UNIQUE(identificador_dispositivo)  
);
```

4.9. log_auditoria

```
CREATE TABLE log_auditoria (  

```

```

id UUID PRIMARY KEY,
organizacao_id UUID,
acao VARCHAR(128) NOT NULL,
tipo_entidade VARCHAR(64),
entidade_id UUID,
decisao VARCHAR(16),
metadados JSONB,
criado_por VARCHAR(64) NOT NULL,
criado_em TIMESTAMP NOT NULL
);
CREATE INDEX idx_auditoria_org_tempo ON log_auditoria(organizacao_id, criado_em);

```

4.10. consentimento

```

CREATE TABLE consentimento (
id UUID PRIMARY KEY,
organizacao_id UUID NOT NULL,
usuario_id VARCHAR(64) NOT NULL,
tipo_consentimento VARCHAR(64) NOT NULL,
status VARCHAR(32) NOT NULL DEFAULT 'ATIVO',
criado_por VARCHAR(64) NOT NULL,
criado_em TIMESTAMP NOT NULL,
atualizado_por VARCHAR(64),
atualizado_em TIMESTAMP,
FOREIGN KEY (organizacao_id) REFERENCES organizacao(id)
);

```

4.11. relacionamento

```

CREATE TABLE relacionamento (
id UUID PRIMARY KEY,
organizacao_id UUID NOT NULL,
sujeito_id VARCHAR(64) NOT NULL,
objeto_id VARCHAR(64) NOT NULL,
tipo_relacionamento VARCHAR(64) NOT NULL,
inicio_validade TIMESTAMP,
fim_validade TIMESTAMP,
criado_por VARCHAR(64) NOT NULL,
criado_em TIMESTAMP NOT NULL,
atualizado_por VARCHAR(64),
atualizado_em TIMESTAMP,
FOREIGN KEY (organizacao_id) REFERENCES organizacao(id)
);
CREATE INDEX idx_relacionamento_org_sujeito ON relacionamento(organizacao_id, sujeito_id);
CREATE INDEX idx_relacionamento_org_objeto ON relacionamento(organizacao_id, objeto_id);

```

4.12. execucao_permissao_usuario

```

CREATE TABLE execucao_permissao_usuario (
id UUID PRIMARY KEY,
organizacao_id UUID NOT NULL,
usuario_id VARCHAR(64) NOT NULL,
recurso VARCHAR(128) NOT NULL,

```

```
escopo VARCHAR(64) NOT NULL,  
condicao_json JSONB,  
criado_por VARCHAR(64) NOT NULL,  
criado_em TIMESTAMP NOT NULL,  
atualizado_por VARCHAR(64),  
atualizado_em TIMESTAMP,  
FOREIGN KEY (organizacao_id) REFERENCES organizacao(id)  
);  
CREATE INDEX idx_permissao_org_usuario ON execucao_permissao_usuario(organizacao_id,  
usuario_id);
```

5. Notas Arquiteturais e Contexto da Implementação Atual

Esta seção documenta o estado atual das escolhas de design aplicadas no esquema físico acima, bem como ausências notáveis que deverão ser tratadas em revisões futuras.

1.Imutabilidade Estrutural: As tabelas registro_presenca e log_auditoria são projetadas conceitualmente para serem de inserção exclusiva (*append-only*). **Nota sobre o schema atual:** Não há *triggers* implementados no script V1 bloqueando ações de UPDATE ou DELETE. A imutabilidade, neste momento, depende exclusivamente das travas na camada da aplicação (Spring Boot).

2.Definições Explícitas e Defaults: As regras padrão foram aplicadas diretamente nas colunas (como DEFAULT CURRENT_TIMESTAMP e *casts* explícitos DEFAULT 'ATIVO'::character varying).

3.Restrições de Domínio (CHECK constraints): Apenas a tabela convite recebeu mapeamento de domínio direto no banco (chk_convite_papel). Para as demais tabelas que possuem status (ex: alocao_unidade, organizacao, registro_presenca), a validação de domínio não está sendo feita no nível de esquema (SQL), mas sim na camada da aplicação.

4.Integridade de Chaves na Tabela alocao_unidade: No script de versão atual (V1), a tabela alocao_unidade foi criada com os tipos de dados necessários, porém **sem a declaração formal de PRIMARY KEY ou FOREIGN KEY** na execução do CREATE TABLE. A junção relacional se mantém de forma lógica pela aplicação, apoiada pelo índice secundário idx_alocacao_vinculo.

5.Hierarquia Organizacional: Foi materializada a decisão da **RFC-0002**, removendo dependência de auto-relacionamento na tabela unidade em favor da tabela relacionamento (modelo de Grafos - ReBAC).

6. Evolução para Schema por Tenant

Estratégias de evolução para schema/banco por locatário (Tier 2 e 3 do projeto) permanecem idênticas àquelas documentadas na **RFC-0004** para garantir o isolamento futuro de grandes clientes Enterprise sem impacto no modelo relacional base exposto neste documento.